SKYNETIX SYSTEMS

# JAVA 25
# COMPLETE PROGRAMMING HANDBOOK

Beginner to advanced • Core Java • Modern Java 25 • JVM • Concurrency • Enterprise development

---

**Concepts explained with coding examples, output walkthroughs and classroom exercises**

A coding-first reference for students, instructors and Java developers

Version 1.0 | November 2026

Prepared for SkyNetix Systems training classes

# How to use this handbook

This handbook is designed to be taught with an IDE and terminal open. Students should type the examples, predict the output, run the program, change one thing at a time and explain the result. Most examples use conventional class-based Java so that learners build strong fundamentals. A dedicated chapter introduces the permanent Java 25 beginner-friendly source-file features.

**Version note:** Java 25 includes three permanent language changes: module import declarations, compact source files and instance main methods, and flexible constructor bodies. Primitive types in patterns, instanceof and switch is a preview feature in Java 25 and must be enabled explicitly. This handbook labels preview syntax separately. Oracle Java 25 language changes.

## Recommended teaching sequence

| Phase | Coverage | Suggested duration |
|---|---|---|
| Java foundation | Chapters 1–10: syntax, OOP, exceptions | 20–24 hours |
| Core APIs | Chapters 11–16: generics, collections, streams, I/O | 16–20 hours |
| Advanced Java | Chapters 17–22: concurrency, JVM, modules, Java 25 | 18–22 hours |
| Application development | Chapters 23–28: JDBC, tests, HTTP, patterns, security | 16–20 hours |
| Project and revision | Chapters 29–30 and capstone | 8–12 hours |

## Learning outcomes

• Write, compile, run and debug Java 25 programs.

• Apply object-oriented design, interfaces, records, sealed classes and generics.

• Use collections, lambdas, streams, Optional and modern date/time APIs.

• Write concurrent programs and understand executors and virtual threads.

• Explain JVM memory, garbage collection, class loading and performance basics.

• Connect Java applications to databases and HTTP services; test code with JUnit.

• Recognize Java 25 language changes and distinguish permanent from preview features.

# Contents

1. Java platform, JDK, JRE, JVM and setup

2. First Java program and compilation

3. Variables, primitive types and type conversion

4. Operators and expressions

5. Conditional statements and switch

6. Loops and iteration

7. Arrays and strings

8. Methods, parameters and recursion

9. Classes, objects, constructors and encapsulation

10. Inheritance, polymorphism and abstraction

11. Interfaces, enums and nested types

12. Exception handling and custom exceptions

13. Generics

14. Collections Framework

15. Lambdas, functional interfaces and method references
16. Stream API and Optional
17. File I/O, NIO.2 and serialization concepts
18. Date and Time API
19. Multithreading and synchronization
20. Executors, CompletableFuture and virtual threads
21. JVM internals, memory and garbage collection
22. Modern Java: records, sealed classes and pattern matching
23. Java 25 language features
24. Modules and modular application design
25. JDBC and database programming
26. Unit testing with JUnit and debugging
27. HTTP Client and REST integration
28. Annotations, reflection and design patterns
29. Performance, security and production practices
30. Capstone: Employee and Order Management API

# 1. Java platform, JDK, JRE, JVM and setup

**Learning objective:** Java source code is compiled into bytecode. The JVM loads and executes bytecode, while the JDK provides development tools such as javac, java, javadoc and jlink.

## Concept explanation

# Verify installation java --version javac --version # Compile and run a class javac Hello.java java Hello

## Coding example

```
// Hello.java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Welcome to Java 25");
    }
}
```

## What to notice

javac translates source into class files. java launches the JVM and invokes the main entry point. The JDK version used to compile and run should be checked in the terminal.

## Second example / implementation pattern

```
// Hello.java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Welcome to Java 25");
    }
}
```

Use the first example as the initial compile/run exercise.

## Classroom exercises

**Exercise 1.** Install a JDK 25 distribution and verify java and javac versions.

**Exercise 2.** Create a folder named java25-training and compile Hello.java.

**Exercise 3.** Run the same program from an IDE and from the terminal.

**Exercise 4.** Use javap -c Hello to inspect bytecode for the simple class.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 2. First Java program and compilation

**Learning objective:** A Java application commonly starts from a class containing public static void main(String[] args). Java 25 also supports compact source files and instance main methods; those are covered later.

## Concept explanation

// Greeting.java public class Greeting { public static void main(String[] args) { String name = "Asha"; System.out.println("Hello, " + name); System.out.printf("Welcome to %s%n", "Java"); } }

## Coding example

```
// Command-line arguments
public class ArgumentsDemo {
    public static void main(String[] args) {
        if (args.length == 0) {
            System.out.println("Please pass your name.");
            return;
        }
        System.out.println("Hello, " + args[0]);
    }
}
// Run: java ArgumentsDemo Ravi
```

## What to notice

System.out.println prints a line. printf formats output. args contains command-line arguments. The source filename should match the public top-level class name.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Print your name, city and course on separate lines.

**Exercise 2.** Use printf to display a product, quantity and price.

**Exercise 3.** Accept two command-line arguments and print a greeting.

**Exercise 4.** Intentionally introduce a missing semicolon, compile and read the compiler error.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 3. Variables, primitive types and type conversion

**Learning objective:** Java is statically typed. Primitive types store simple values; reference variables refer to objects. Local variables must be initialized before use.

## Concept explanation

public class TypesDemo { public static void main(String[] args) { int quantity = 4; long population = 8_000_000_000L; double price = 149.95; char grade = 'A'; boolean active = true; String course = "Java 25"; int total = quantity * 10; double tax = price * 0.18; System.out.println(total); System.out.println(tax); System.out.println(course + " " + grade + " " + active); } }

## Coding example

```
public class ConversionDemo {
    public static void main(String[] args) {
        int whole = (int) 19.99; // narrowing: decimal part is lost
        double precise = whole; // widening
        String text = "250";
        int amount = Integer.parseInt(text);

        System.out.println(whole);   // 19
        System.out.println(precise); // 19.0
        System.out.println(amount + 50); // 300
    }
}
```

## What to notice

Use int for ordinary whole numbers, long for larger integer ranges, double for general floating-point calculations, boolean for true/false and char for a UTF-16 code unit. Use BigDecimal for decimal monetary calculations that require decimal precision.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Declare one variable of every primitive type and print it.

**Exercise 2.** Convert a numeric String to int and handle NumberFormatException.

**Exercise 3.** Calculate a shopping bill and explain why double can be unsuitable for exact money.

**Exercise 4.** Use BigDecimal to add two decimal amounts without binary floating-point surprises.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 4. Operators and expressions

**Learning objective:** Operators perform arithmetic, comparison, logical evaluation, assignment and bit-level operations. Understand precedence and use parentheses to make intent clear.

## Concept explanation

public class OperatorsDemo { public static void main(String[] args) { int a = 17, b = 5; System.out.println(a + b); // 22 System.out.println(a / b); // 3: integer division System.out.println(a % b); // 2 System.out.println(a > b && b > 0); // true int score = 75; String result = score >= 40 ? "Pass" : "Fail"; System.out.println(result); } }

## Coding example

```
public class ShortCircuitDemo {
    public static void main(String[] args) {
        String value = null;
        if (value != null && value.length() > 0) {
            System.out.println(value);
        } else {
            System.out.println("No value");
        }
    }
}
```

## What to notice

&& and || use short-circuit evaluation. In the example, the second condition is not evaluated when value is null, preventing a NullPointerException.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Build a calculator for +, -, *, / and %.

**Exercise 2.** Check whether a number is within a range using logical operators.

**Exercise 3.** Demonstrate the difference between == and equals for String values.

**Exercise 4.** Use bitwise AND to test whether a number is even or odd, then explain the result.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 5. Conditional statements and switch

**Learning objective:** if/else is suitable for ranges and complex conditions. switch is useful when selecting among discrete alternatives. Java supports switch expressions that return values.

## Concept explanation

public class GradeDemo { public static void main(String[] args) { int marks = 82; if (marks >= 90) { System.out.println("A"); } else if (marks >= 75) { System.out.println("B"); } else if (marks >= 60) { System.out.println("C"); } else { System.out.println("Needs improvement"); } } }

## Coding example

```
public class SwitchDemo {
    public static void main(String[] args) {
        int day = 6;
        String type = switch (day) {
            case 1, 7 -> "Weekend";
            case 2, 3, 4, 5, 6 -> "Weekday";
            default -> "Invalid";
        };
        System.out.println(type);
    }
}
```

## What to notice

Arrow labels do not fall through. A switch expression must produce a value on every path, including a default or exhaustive set of cases.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Build a menu-driven calculator using switch.

**Exercise 2.** Classify an employee bonus by performance rating.

**Exercise 3.** Use a switch expression to map month number to month name.

**Exercise 4.** Handle an invalid menu choice and keep the program running until Exit is selected.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 6. Loops and iteration

**Learning objective:** for is commonly used when the number of iterations is known. while and do-while are useful when repetition depends on a condition. Enhanced for iterates over arrays and collections.

## Concept explanation

public class LoopDemo { public static void main(String[] args) { for (int i = 1; i <= 5; i++) { System.out.println("Count: " + i); } int n = 5; int factorial = 1; while (n > 1) { factorial *= n; n--; } System.out.println("Factorial: " + factorial); } }

## Coding example

```
public class PrimeCheck {
    public static void main(String[] args) {
        int number = 29;
        boolean prime = number > 1;
        for (int divisor = 2; divisor * divisor <= number; divisor++) {
            if (number % divisor == 0) {
                prime = false;
                break;
            }
        }
        System.out.println(prime ? "Prime" : "Not prime");
    }
}
```

## What to notice

break exits a loop. continue skips the remaining statements in the current iteration. Avoid loops with conditions that never change.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Print a multiplication table.

**Exercise 2.** Calculate factorial using a for loop and while loop.

**Exercise 3.** Print all prime numbers between 1 and 200.

**Exercise 4.** Create a number-guessing game with a maximum number of attempts.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 7. Arrays and strings

**Learning objective:** Arrays have a fixed length and store elements of one declared type. String is immutable; StringBuilder is useful for repeated modifications.

## Concept explanation

import java.util.Arrays; public class ArrayDemo { public static void main(String[] args) { int[] marks = {78, 91, 84, 66}; int sum = 0; for (int mark : marks) sum += mark; double average = (double) sum / marks.length; Arrays.sort(marks); System.out.println(Arrays.toString(marks)); System.out.println("Average: " + average); } }

## Coding example

```
public class StringDemo {
    public static void main(String[] args) {
        String text = "  Java 25 Training  ";
        String cleaned = text.trim().toUpperCase();
        StringBuilder builder = new StringBuilder();
        for (int i = 1; i <= 3; i++) {
            builder.append("Lab ").append(i).append(System.lineSeparator());
        }
        System.out.println(cleaned);
        System.out.println(builder);
    }
}
```

## What to notice

String methods return new strings; they do not modify the original. For Unicode-aware text processing, be aware that char represents a UTF-16 code unit, not necessarily a complete Unicode code point.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Find minimum, maximum and average in an integer array.

**Exercise 2.** Reverse a string using StringBuilder.

**Exercise 3.** Count vowels, consonants, digits and spaces in a sentence.

**Exercise 4.** Check whether a phrase is a palindrome after removing spaces and punctuation.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 8. Methods, parameters and recursion

**Learning objective:** Methods define reusable behavior. Java passes arguments by value; for object references, the copied value is the reference. Method overloading uses different parameter lists.

## Concept explanation

public class MethodDemo { static int add(int a, int b) { return a + b; } static double add(double a, double b) { return a + b; } public static void main(String[] args) { System.out.println(add(10, 20)); System.out.println(add(2.5, 3.75)); } }

## Coding example

```
public class RecursionDemo {
    static long factorial(int n) {
        if (n < 0) throw new IllegalArgumentException("n must be >= 0");
        if (n <= 1) return 1;
        return n * factorial(n - 1);
    }

    public static void main(String[] args) {
        System.out.println(factorial(5)); // 120
    }
}
```

## What to notice

Recursion requires a base case and a progress step. Deep recursion can cause StackOverflowError; iteration is often preferable for simple repeated work.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Write methods for addition, subtraction, multiplication and division.

**Exercise 2.** Create overloaded methods to calculate area of a circle and rectangle.

**Exercise 3.** Write recursive and iterative factorial implementations.

**Exercise 4.** Explain with a small experiment why Java is pass-by-value.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 9. Classes, objects, constructors and encapsulation

**Learning objective:** A class defines state and behavior. An object is an instance. Encapsulation protects invariants by keeping fields private and exposing controlled methods.

## Concept explanation

public class BankAccount { private final String accountNumber; private long balanceInPaise; public BankAccount(String accountNumber, long openingBalanceInPaise) { if (accountNumber == null || accountNumber.isBlank()) { throw new IllegalArgumentException("Account number required"); } if (openingBalanceInPaise < 0) { throw new IllegalArgumentException("Opening balance cannot be negative"); } this.accountNumber = accountNumber; this.balanceInPaise = openingBalanceInPaise; } public void deposit(long amountInPaise) { if (amountInPaise <= 0) throw new IllegalArgumentException("Amount must be positive"); balanceInPaise += amountInPaise; } public long getBalanceInPaise() { return balanceInPaise; } }

## Coding example

```
public class BankDemo {
    public static void main(String[] args) {
        BankAccount account = new BankAccount("SB1001", 10_000);
        account.deposit(2_500);
        System.out.println(account.getBalanceInPaise());
    }
}
```

## What to notice

Fields are private, and the constructor validates the object's initial state. A real financial system would also need transaction safety, persistence, authorization and audit logging.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Create an Employee class with private fields and validation.

**Exercise 2.** Add constructors for default and parameterized initialization.

**Exercise 3.** Create an immutable Address class.

**Exercise 4.** Implement a LibraryBook class with borrow and return methods that protect its availability state.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 10. Inheritance, polymorphism and abstraction

**Learning objective:** Inheritance models an is-a relationship. Runtime polymorphism allows code to use a superclass or interface reference while invoking the actual object's overridden implementation. Abstract classes can contain shared state and behavior.

## Concept explanation

abstract class Shape { abstract double area(); void printArea() { System.out.println("Area = " + area()); } } class Circle extends Shape { private final double radius; Circle(double radius) { if (radius <= 0) throw new IllegalArgumentException(); this.radius = radius; } @Override double area() { return Math.PI * radius * radius; } } public class ShapeDemo { public static void main(String[] args) { Shape shape = new Circle(3); shape.printArea(); } }

## Coding example

```
class Employee {
    String name;
    Employee(String name) { this.name = name; }
    double calculatePay() { return 0; }
}
class SalariedEmployee extends Employee {
    private final double monthlySalary;
    SalariedEmployee(String name, double salary) {
        super(name);
        this.monthlySalary = salary;
    }
    @Override double calculatePay() { return monthlySalary; }
}
```

## What to notice

Override methods to specialize behavior. Use @Override so the compiler catches signature mistakes. Prefer composition over inheritance when the relationship is not genuinely is-a.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Create Shape, Circle and Rectangle classes and calculate areas polymorphically.

**Exercise 2.** Create Employee subclasses for salaried and hourly employees.

**Exercise 3.** Demonstrate method overriding and explain dynamic dispatch.

**Exercise 4.** Refactor an inheritance-heavy design into composition where appropriate.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 11. Interfaces, enums and nested types

**Learning objective:** An interface defines a contract. A class can implement multiple interfaces. Enums represent a fixed set of values and can contain fields and methods.

## Concept explanation

interface Payable { long amountInPaise(); } record Invoice(String id, long amountInPaise) implements Payable {} public class InterfaceDemo { static void printPayment(Payable payable) { System.out.println(payable.amountInPaise()); } public static void main(String[] args) { printPayment(new Invoice("INV-1", 125_000)); } }

## Coding example

```
enum OrderStatus {
    CREATED, PAID, PACKED, SHIPPED, CANCELLED
}

public class EnumDemo {
    public static void main(String[] args) {
        OrderStatus status = OrderStatus.PAID;
        switch (status) {
            case CREATED -> System.out.println("Await payment");
            case PAID -> System.out.println("Prepare order");
            case PACKED -> System.out.println("Ready to ship");
            case SHIPPED -> System.out.println("In transit");
            case CANCELLED -> System.out.println("Order cancelled");
        }
    }
}
```

## What to notice

Interfaces enable loose coupling and testability. Enums prevent invalid string-based states. Nested classes are useful when a type is closely tied to its enclosing type.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Define a PaymentMethod interface and implement CardPayment and UpiPayment.

**Exercise 2.** Create an enum for order status and use it in a workflow.

**Exercise 3.** Use a functional interface and implement it with an anonymous class.

**Exercise 4.** Design a notification service against an interface rather than a concrete implementation.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 12. Exception handling and custom exceptions

**Learning objective:** Exceptions represent abnormal conditions. Checked exceptions must be caught or declared; unchecked exceptions usually represent programming errors or invalid arguments. Use try-with-resources for closeable resources.

## Concept explanation

import java.io.BufferedReader; import java.io.FileReader; import java.io.IOException; public class FileReadDemo { public static void main(String[] args) { try (BufferedReader reader = new BufferedReader(new FileReader("input.txt"))) { System.out.println(reader.readLine()); } catch (IOException ex) { System.err.println("Could not read file: " + ex.getMessage()); } } }

## Coding example

```
class InsufficientFundsException extends Exception {
    InsufficientFundsException(String message) { super(message); }
}
class Wallet {
    private int balance = 500;
    void withdraw(int amount) throws InsufficientFundsException {
        if (amount > balance) {
            throw new InsufficientFundsException("Balance too low");
        }
        balance -= amount;
    }
}
```

## What to notice

Catch exceptions at a level that can recover or add useful context. Do not swallow exceptions with empty catch blocks. Preserve the cause when wrapping an exception.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Handle invalid numeric input using NumberFormatException.

**Exercise 2.** Create a custom checked exception for insufficient inventory.

**Exercise 3.** Use try-with-resources to read a file safely.

**Exercise 4.** Create a method that throws an exception and document when callers should handle it.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 13. Generics

**Learning objective:** Generics provide compile-time type safety and reduce casting. Type parameters are commonly named T, E, K and V. Java generics use type erasure at runtime.

## Concept explanation

import java.util.ArrayList; import java.util.List; public class GenericsDemo { public static void main(String[] args) { List<String> names = new ArrayList<>(); names.add("Asha"); names.add("Ravi"); String first = names.getFirst(); // Java 21+ SequencedCollection API System.out.println(first); } }

## Coding example

```
class Box<T> {
    private T value;
    void set(T value) { this.value = value; }
    T get() { return value; }
}
public class BoxDemo {
    public static void main(String[] args) {
        Box<Integer> box = new Box<>();
        box.set(42);
        System.out.println(box.get());
    }
}
```

## What to notice

Use bounded type parameters such as <T extends Number> when an operation requires a capability. Avoid raw types because they bypass generic type checking.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Create a generic Pair class.

**Exercise 2.** Write a generic method that prints any array.

**Exercise 3.** Create a repository interface Repository.

**Exercise 4.** Experiment with extends and super wildcards and explain PECS: producer extends, consumer super.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 14. Collections Framework

**Learning objective:** Collections provide dynamic data structures. List preserves order and allows duplicates; Set enforces uniqueness; Map associates keys with values; Queue models processing order.

## Concept explanation

import java.util.*; public class CollectionsDemo { public static void main(String[] args) { List<String> names = new ArrayList<>(List.of("Ravi", "Asha", "Ravi")); Set<String> unique = new LinkedHashSet<>(names); Map<String, Integer> scores = new HashMap<>(); scores.put("Asha", 91); scores.merge("Asha", 5, Integer::sum); System.out.println(names); System.out.println(unique); System.out.println(scores); } }

## Coding example

```
import java.util.PriorityQueue;
public class QueueDemo {
    public static void main(String[] args) {
        PriorityQueue<Integer> jobs = new PriorityQueue<>();
        jobs.offer(30);
        jobs.offer(10);
        jobs.offer(20);
        while (!jobs.isEmpty()) {
            System.out.println(jobs.poll());
        }
    }
}
```

## What to notice

Choose a collection based on access pattern and ordering requirements. HashMap does not guarantee iteration order. TreeMap sorts keys. Concurrent collections are designed for concurrent access.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Count word frequency using HashMap.

**Exercise 2.** Remove duplicates while preserving insertion order.

**Exercise 3.** Use ArrayDeque to implement a stack and queue.

**Exercise 4.** Compare ArrayList and LinkedList for indexed access and insertion patterns.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 15. Lambdas, functional interfaces and method references

**Learning objective:** A functional interface has one abstract method. Lambdas provide concise implementations. Method references reuse an existing method as a lambda target.

## Concept explanation

import java.util.function.Predicate; import java.util.function.Function; public class LambdaDemo { public static void main(String[] args) { Predicate<String> longName = name -> name.length() > 4; Function<String, String> upper = String::toUpperCase; System.out.println(longName.test("Akhil")); System.out.println(upper.apply("java")); } }

## Coding example

```
import java.util.List;
public class MethodReferenceDemo {
    public static void main(String[] args) {
        List<String> names = List.of("Ravi", "Asha", "Meera");
        names.stream()
             .map(String::toUpperCase)
             .forEach(System.out::println);
    }
}
```

## What to notice

Common functional interfaces include Predicate<T>, Function<T,R>, Consumer<T>, Supplier<T> and UnaryOperator<T>. Lambdas capture local variables only when they are final or effectively final.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Sort employees by salary using a lambda comparator.

**Exercise 2.** Use Predicate to filter active users.

**Exercise 3.** Use Function to convert a list of names to uppercase.

**Exercise 4.** Create a custom functional interface and pass a lambda to a method.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 16. Stream API and Optional

**Learning objective:** Streams describe data-processing pipelines. Intermediate operations are lazy; terminal operations trigger processing. Streams should generally be used once. Optional models a possibly absent result.

## Concept explanation

import java.util.List; record Employee(String name, String department, int salary) {} public class StreamDemo { public static void main(String[] args) { List<Employee> employees = List.of( new Employee("Asha", "IT", 90000), new Employee("Ravi", "HR", 60000), new Employee("Meera", "IT", 110000) ); double average = employees.stream() .filter(e -> e.department().equals("IT")) .mapToInt(Employee::salary) .average() .orElse(0.0); System.out.println(average); } }

## Coding example

```
import java.util.Optional;
public class OptionalDemo {
    static Optional<String> findEmail(String user) {
        if ("asha".equals(user)) return Optional.of("asha@example.test");
        return Optional.empty();
    }
    public static void main(String[] args) {
        String email = findEmail("asha")
            .map(String::toLowerCase)
            .orElse("not-found@example.test");
        System.out.println(email);
    }
}
```

## What to notice

Use map to transform, filter to retain matching values, flatMap to flatten nested containers, collect to gather results and reduce for aggregation. Avoid Optional.get without checking presence.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Filter employees by department and salary.

**Exercise 2.** Group employees by department using Collectors.groupingBy.

**Exercise 3.** Find the highest-paid employee using max and Comparator.

**Exercise 4.** Use Optional to model a user lookup that may return no result.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 17. File I/O, NIO.2 and serialization concepts

**Learning objective:** java.nio.file provides Path and Files APIs for modern file operations. Serialization should be treated carefully; do not deserialize untrusted data.

## Concept explanation

import java.nio.file.*; import java.io.IOException; import java.util.List; public class FilesDemo { public static void main(String[] args) throws IOException { Path path = Path.of("notes.txt"); Files.write(path, List.of("Java 25", "NIO.2", "File operations")); List<String> lines = Files.readAllLines(path); lines.forEach(System.out::println); } }

## Coding example

```
import java.nio.file.*;
import java.io.IOException;

public class WalkDemo {
    public static void main(String[] args) throws IOException {
        try (var paths = Files.walk(Path.of("."), 2)) {
            paths.filter(Files::isRegularFile)
                 .forEach(System.out::println);
        }
    }
}
```

## What to notice

Use buffered APIs for large files and stream-based file operations when appropriate. Close streams. Validate paths when accepting file names from users to prevent path traversal.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Read a CSV file line by line and count records.

**Exercise 2.** Write a report to a file using Files.writeString.

**Exercise 3.** List all .java files below a directory.

**Exercise 4.** Validate a user-provided path so it cannot escape an allowed folder.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 18. Date and Time API

**Learning objective:** The java.time API is immutable and thread-safe. LocalDate represents a date without a time zone; Instant represents a point on the UTC timeline; ZonedDateTime includes a zone.

## Concept explanation

import java.time.*; import java.time.format.DateTimeFormatter; public class DateTimeDemo { public static void main(String[] args) { LocalDate today = LocalDate.now(); LocalDate dueDate = today.plusDays(14); Instant now = Instant.now(); ZonedDateTime puneTime = now.atZone(ZoneId.of("Asia/Kolkata")); System.out.println(today); System.out.println(dueDate); System.out.println(puneTime.format(DateTimeFormatter.ISO_ZONED_DATE_TIME)); } }

## Coding example

```
import java.time.*;
public class DurationDemo {
    public static void main(String[] args) {
        LocalDate start = LocalDate.of(2026, 1, 1);
        LocalDate end = LocalDate.of(2026, 12, 31);
        Period period = Period.between(start, end);
        System.out.println(period.getMonths() + " months");
        System.out.println(Duration.ofMinutes(90).toHours());
    }
}
```

## What to notice

Use Instant for timestamps, LocalDate for date-only business concepts, and ZonedDateTime when a time zone matters. Avoid relying on the machine's default zone for business-critical scheduling.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Calculate employee tenure using Period.

**Exercise 2.** Convert an Instant to Asia/Kolkata time.

**Exercise 3.** Parse and format dates using DateTimeFormatter.

**Exercise 4.** Calculate the number of business days between two dates, excluding weekends.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 19. Multithreading and synchronization

**Learning objective:** Threads allow tasks to run concurrently. Shared mutable state introduces race conditions. Synchronization and atomic classes help coordinate access.

## Concept explanation

public class ThreadDemo { public static void main(String[] args) throws InterruptedException { Thread worker = new Thread(() -> { for (int i = 1; i <= 3; i++) { System.out.println("Worker " + i); } }); worker.start(); worker.join(); System.out.println("Main finished"); } }

## Coding example

```
class Counter {
    private int value;
    public synchronized void increment() { value++; }
    public synchronized int get() { return value; }
}
public class CounterDemo {
    public static void main(String[] args) throws InterruptedException {
        Counter counter = new Counter();
        Thread a = new Thread(() -> { for (int i=0;i<10000;i++) counter.increment(); });
        Thread b = new Thread(() -> { for (int i=0;i<10000;i++) counter.increment(); });
        a.start(); b.start(); a.join(); b.join();
        System.out.println(counter.get()); // 20000
    }
}
```

## What to notice

start schedules a new thread; calling run directly does not start a new thread. synchronized provides mutual exclusion and visibility guarantees for the protected monitor. Never assume thread execution order.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Run two threads that print numbered messages and observe interleaving.

**Exercise 2.** Demonstrate a race condition with an unsynchronized counter.

**Exercise 3.** Fix the counter using synchronized and then AtomicInteger.

**Exercise 4.** Use a thread-safe collection to share results between workers.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 20. Executors, CompletableFuture and virtual threads

**Learning objective:** Executors manage task execution. CompletableFuture composes asynchronous operations. Virtual threads make thread-per-task designs practical for many blocking I/O workloads; they do not make CPU-bound work faster.

## Concept explanation

import java.util.concurrent.*; public class ExecutorDemo { public static void main(String[] args) throws Exception { try (ExecutorService pool = Executors.newFixedThreadPool(2)) { Future<Integer> result = pool.submit(() -> 20 + 22); System.out.println(result.get()); } } }

## Coding example

```
import java.util.concurrent.*;

public class VirtualThreadDemo {
    public static void main(String[] args) throws InterruptedException {
        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
            for (int i = 0; i < 5; i++) {
                int taskId = i;
                executor.submit(() ->
                    System.out.println("Task " + taskId + " on " + Thread.currentThread()));
            }
        }
    }
}
```

## What to notice

Use bounded platform-thread pools for CPU-intensive work. Virtual threads suit large numbers of mostly-blocking tasks, subject to resource limits in databases and downstream services. Avoid pooling virtual threads as if they were scarce platform threads.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Submit Callable tasks to an ExecutorService and collect Future results.

**Exercise 2.** Combine two CompletableFuture operations using thenCombine.

**Exercise 3.** Launch many simulated I/O tasks using virtual threads.

**Exercise 4.** Add timeout and exception handling to an asynchronous pipeline.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 21. JVM internals, memory and garbage collection

**Learning objective:** The JVM runtime includes class loading, runtime data areas, execution engine and garbage collection. Objects are commonly allocated on the heap; each thread has a stack of frames. Exact implementation details vary by JVM.

## Concept explanation

public class MemoryDemo { static class Customer { String name; Customer(String name) { this.name = name; } } public static void main(String[] args) { Customer first = new Customer("Asha"); Customer second = first; second.name = "Ravi"; System.out.println(first.name); // Ravi } }

## Coding example

```
// Useful diagnostic commands (run from a terminal)
java -Xms256m -Xmx512m MemoryDemo
jcmd <pid> VM.flags
jcmd <pid> GC.heap_info
jcmd <pid> Thread.print
jcmd <pid> JFR.start duration=60s filename=recording.jfr
```

## What to notice

Understand object reachability, allocation rate, heap sizing, garbage-collection pauses and thread dumps. Do not call System.gc() as a routine memory-management strategy.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Draw stack and heap references for a small object graph.

**Exercise 2.** Run an allocation loop and inspect memory behavior with JFR or VisualVM.

**Exercise 3.** Capture and inspect a thread dump.

**Exercise 4.** Explain the difference between memory leak, high allocation rate and normal garbage collection.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 22. Modern Java: records, sealed classes and pattern matching

**Learning objective:** Records are concise data carriers. Sealed classes restrict permitted subtypes. Pattern matching can simplify type checks and switch logic.

## Concept explanation

record Customer(String id, String name) {} public class RecordDemo { public static void main(String[] args) { Customer customer = new Customer("C1", "Asha"); System.out.println(customer.name()); System.out.println(customer); } }

## Coding example

```
sealed interface Payment permits CardPayment, CashPayment {}
record CardPayment(String lastFour) implements Payment {}
record CashPayment(int amountInPaise) implements Payment {}

public class PatternDemo {
    static String describe(Payment payment) {
        return switch (payment) {
            case CardPayment card -> "Card ending " + card.lastFour();
            case CashPayment cash -> "Cash " + cash.amountInPaise();
        };
    }
    public static void main(String[] args) {
        System.out.println(describe(new CardPayment("1234")));
    }
}
```

## What to notice

Records generate canonical constructors, accessors, equals, hashCode and toString. They are shallowly immutable: mutable components still need defensive copying. Sealed hierarchies make domain variants explicit.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Create a Product record with validation in a compact constructor.

**Exercise 2.** Model payment types using a sealed interface.

**Exercise 3.** Use pattern matching for instanceof to extract a typed value.

**Exercise 4.** Create a switch expression over a sealed hierarchy and test every permitted subtype.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 23. Java 25 language features

**Learning objective:** Java 25 finalizes three language features: module import declarations (JEP 511), compact source files and instance main methods (JEP 512), and flexible constructor bodies (JEP 513). Primitive types in patterns, instanceof and switch is a third-preview feature (JEP 507).

## Concept explanation

// Compact source file: Hello.java void main() { IO.println("Hello from Java 25"); } // Run with: java Hello.java

## Coding example

```
// Module import declaration
import module java.base;

public class ModuleImportDemo {
    public static void main(String[] args) {
        List<String> names = List.of("Asha", "Ravi");
        names.forEach(System.out::println);
    }
}
```

## What to notice

Compact source files reduce ceremony for small programs. Module import declarations import the packages exported by a module. For regular enterprise applications, conventional classes and explicit imports may still improve clarity. Flexible constructor bodies allow safe validation before an explicit super(...) or this(...) call, subject to restrictions on referencing the object under construction.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Run a compact source file using Java 25 and then convert it to a conventional class.

**Exercise 2.** Use import module java.base; in a small demonstration and identify which types become available.

**Exercise 3.** Write a constructor that validates an argument before calling super(argument).

**Exercise 4.** Explore JEP 507 primitive pattern matching only in a separate preview-feature exercise using --enable-preview --release 25; label it as preview, not standard production syntax.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 24. Modules and modular application design

**Learning objective:** The Java Platform Module System groups packages into named modules. module-info.java declares module identity and dependencies. exports exposes packages; requires declares dependencies.

## Concept explanation

// src/com.example.greetings/module-info.java module com.example.greetings { exports com.example.greetings; } // src/com.example.greetings/com/example/greetings/Greeter.java package com.example.greetings; public class Greeter { public String greet(String name) { return "Hello " + name; } }

## Coding example

```
// Compile a simple module source tree
javac -d out --module-source-path src -m com.example.greetings
java --module-path out -m com.example.greetings/com.example.greetings.Main
```

## What to notice

Modules can improve encapsulation and dependency boundaries. A package is not automatically exported just because it is inside a module.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Create a module with one exported package.

**Exercise 2.** Create a second module that requires the first.

**Exercise 3.** Remove an export and observe the compile-time access error.

**Exercise 4.** Document which packages should be public API and which should remain internal.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 25. JDBC and database programming

**Learning objective:** JDBC provides a standard API for relational database access. Use PreparedStatement for parameterized SQL, try-with-resources for resource cleanup and transactions for multi-step changes.

## Concept explanation

import java.sql.*; public class JdbcReadDemo { public static void main(String[] args) throws SQLException { String url = "jdbc:postgresql://localhost:5432/training"; String user = System.getenv("DB_USER"); String password = System.getenv("DB_PASSWORD"); String sql = "SELECT id, name FROM employee WHERE department = ?"; try (Connection con = DriverManager.getConnection(url, user, password); PreparedStatement ps = con.prepareStatement(sql)) { ps.setString(1, "Engineering"); try (ResultSet rs = ps.executeQuery()) { while (rs.next()) { System.out.println(rs.getLong("id") + " " + rs.getString("name")); } } } }

## Coding example

```
// Transaction outline
try (Connection con = dataSource.getConnection()) {
    con.setAutoCommit(false);
    try {
        // execute parameterized debit update
        // execute parameterized credit update
        con.commit();
    } catch (SQLException ex) {
        con.rollback();
        throw ex;
    }
}
```

## What to notice

Use a connection pool in server applications rather than opening a new physical connection for every request. Never concatenate untrusted input into SQL. Store database credentials outside source code.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Create an employee table in a local training database.

**Exercise 2.** Insert and retrieve employees using PreparedStatement.

**Exercise 3.** Update a record and handle the case where no row was changed.

**Exercise 4.** Transfer funds between two accounts in a transaction and test rollback.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 26. Unit testing with JUnit and debugging

**Learning objective:** Unit tests verify behavior at small boundaries. Tests should be repeatable and independent. JUnit 5 provides annotations and assertions; Mockito can isolate collaborators when needed.

## Concept explanation

import org.junit.jupiter.api.Test; import static org.junit.jupiter.api.Assertions.*; class Calculator { int add(int a, int b) { return a + b; } } class CalculatorTest { @Test void addsTwoNumbers() { assertEquals(5, new Calculator().add(2, 3)); } }

## Coding example

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class Account {
    void withdraw(int amount) {
        if (amount <= 0) throw new IllegalArgumentException();
    }
}
class AccountTest {
    @Test
    void rejectsNonPositiveWithdrawal() {
        Account account = new Account();
        assertThrows(IllegalArgumentException.class,
                     () -> account.withdraw(0));
    }
}
```

## What to notice

Test behavior, not private implementation details. Include normal cases, boundaries and failure cases. Use debugger breakpoints, watches and call stacks to inspect unexpected behavior.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Create JUnit tests for a calculator including divide-by-zero.

**Exercise 2.** Test a bank account's deposit and withdrawal invariants.

**Exercise 3.** Use parameterized tests for multiple invalid inputs.

**Exercise 4.** Debug a deliberately failing test and document the root cause.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 27. HTTP Client and REST integration

**Learning objective:** Java's HttpClient supports synchronous and asynchronous HTTP requests. Validate response status and parse the response format using a suitable JSON library in real applications.

## Concept explanation

import java.net.URI; import java.net.http.*; public class HttpClientDemo { public static void main(String[] args) throws Exception { HttpClient client = HttpClient.newHttpClient(); HttpRequest request = HttpRequest.newBuilder() .uri(URI.create("https://httpbin.org/get")) .GET() .build(); HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString()); System.out.println(response.statusCode()); System.out.println(response.body()); } }

## Coding example

```
import java.net.URI;
import java.net.http.*;
import java.time.Duration;

public class HttpPostDemo {
    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newBuilder()
            .connectTimeout(Duration.ofSeconds(5))
            .build();

        String json = "{\"name\":\"Demo\"}";
        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("https://httpbin.org/post"))
            .header("Content-Type", "application/json")
            .timeout(Duration.ofSeconds(10))
            .POST(HttpRequest.BodyPublishers.ofString(json))
            .build();

        HttpResponse<String> response =
            client.send(request, HttpResponse.BodyHandlers.ofString());
        System.out.println(response.statusCode());
    }
}
```

## What to notice

Production clients need timeouts, retry policies appropriate to the HTTP method, authentication, response validation, observability and careful handling of personal data. GET is generally safe; retrying a POST may duplicate an action unless idempotency is designed.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Call a public test GET endpoint and inspect status and headers.

**Exercise 2.** Send a JSON POST request to a test endpoint.

**Exercise 3.** Implement a timeout and handle interruption correctly.

**Exercise 4.** Create a small API client class and unit-test its response mapping separately from network calls.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 28. Annotations, reflection and design patterns

**Learning objective:** Annotations attach metadata to program elements. Reflection inspects types at runtime. Design patterns provide reusable approaches to recurring design problems.

## Concept explanation

import java.lang.annotation.*; @Retention(RetentionPolicy.RUNTIME) @Target(ElementType.TYPE) @interface Audited {} @Audited class PaymentService {} public class AnnotationDemo { public static void main(String[] args) { boolean present = PaymentService.class.isAnnotationPresent(Audited.class); System.out.println(present); } }

## Coding example

```
interface MessageSender { void send(String message); }
class EmailSender implements MessageSender {
    public void send(String message) { System.out.println("Email: " + message); }
}
class NotificationService {
    private final MessageSender sender;
    NotificationService(MessageSender sender) { this.sender = sender; }
    void notifyUser(String text) { sender.send(text); }
}
```

## What to notice

Dependency injection is a form of inversion of control. Strategy allows interchangeable behavior. Factory centralizes object creation. Builder helps construct complex immutable objects. Use reflection sparingly because it can weaken encapsulation and complicate debugging.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Create a custom runtime annotation and inspect it using reflection.

**Exercise 2.** Implement Strategy with two discount algorithms.

**Exercise 3.** Implement a simple Factory for payment processors.

**Exercise 4.** Use Builder to create an immutable configuration object.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 29. Performance, security and production practices

**Learning objective:** Correctness comes first. Measure before optimizing. Production Java systems need secure input handling, bounded resource usage, observability, dependency hygiene and clear failure behavior.

## Concept explanation

// Avoid building a large string with repeated concatenation in a loop StringBuilder report = new StringBuilder(); for (int i = 1; i <= 1000; i++) { report.append("Record ").append(i).append(System.lineSeparator()); } System.out.println(report.length());

## Coding example

```
// Defensive input validation example
public class Validation {
    static String normalizeEmail(String email) {
        if (email == null || email.isBlank()) {
            throw new IllegalArgumentException("Email is required");
        }
        String normalized = email.trim().toLowerCase(java.util.Locale.ROOT);
        if (!normalized.matches("^[^\\s@]+@[^\\s@]+\\.[^\\s@]+$")) {
            throw new IllegalArgumentException("Invalid email format");
        }
        return normalized;
    }
}
```

## What to notice

Use JFR, profilers and metrics to locate real bottlenecks. Avoid logging secrets. Validate and encode untrusted input for its context. Keep dependencies updated and scan them. Apply least privilege to database and service accounts.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Profile a simple program and identify a measurable bottleneck.

**Exercise 2.** Review a sample service for unbounded thread creation and unbounded queues.

**Exercise 3.** Create a secure configuration checklist for a Java service.

**Exercise 4.** Add structured logging with correlation IDs while ensuring passwords and tokens are not logged.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# 30. Capstone: Employee and Order Management API

**Learning objective:** Combine object-oriented design, collections, validation, persistence, testing and HTTP concepts in one project. This chapter is a blueprint; students can implement the persistence and REST layers with JDBC and a framework such as Spring Boot.

## Concept explanation

// Domain model example import java.math.BigDecimal; import java.util.Objects; public record Employee( long id, String name, String department, BigDecimal salary ) { public Employee { if (id <= 0) throw new IllegalArgumentException("id must be positive"); if (name == null || name.isBlank()) throw new IllegalArgumentException("name required"); Objects.requireNonNull(salary, "salary required"); if (salary.signum() < 0) throw new IllegalArgumentException("salary cannot be negative"); } }

## Coding example

```
import java.util.*;
public class EmployeeService {
    private final Map<Long, Employee> employees = new HashMap<>();

    public Employee create(Employee employee) {
        if (employees.containsKey(employee.id())) {
            throw new IllegalStateException("Employee already exists");
        }
        employees.put(employee.id(), employee);
        return employee;
    }

    public Optional<Employee> findById(long id) {
        return Optional.ofNullable(employees.get(id));
    }

    public List<Employee> findByDepartment(String department) {
        return employees.values().stream()
            .filter(e -> e.department().equalsIgnoreCase(department))
            .toList();
    }
}
```

## What to notice

Start with an in-memory service and unit tests. Then replace the repository implementation with JDBC. Add a REST layer only after the domain behavior is tested. Keep controller, service and repository responsibilities separate.

## Second example / implementation pattern

## Classroom exercises

**Exercise 1.** Implement Employee create, find, update and delete operations with validation.

**Exercise 2.** Add Order and OrderItem records and calculate totals using BigDecimal.

**Exercise 3.** Persist employees and orders using JDBC and transactions.

**Exercise 4.** Expose REST endpoints, add JUnit tests, logging, error responses and a README with run instructions.

## Instructor checkpoint

Ask the learner to run the code, explain the key statements, modify one input, predict the new output and demonstrate one invalid or boundary case.

# Java 25 feature notes and preview-feature discipline

Java 25's permanent language changes are useful in teaching and small utilities, but do not replace the conventional class-based structure used by many enterprise projects.

## JEP 512 — Compact source files and instance main methods

```
void main() {
    IO.println("Start with a simple Java 25 program");
}
```

This is a compact source file. The Java launcher can run a source file directly with java Hello.java. The feature supports a smoother learning path from simple code to conventional classes.

## JEP 511 — Module import declarations

```
import module java.base;

public class ImportsExample {
    public static void main(String[] args) {
        List<String> values = List.of("one", "two");
        values.forEach(System.out::println);
    }
}
```

A module import declaration imports the packages exported by a module. Use it when it improves clarity; explicit imports remain common in team code.

## JEP 513 — Flexible constructor bodies

```
class PositiveValue {
    private final int value;

    PositiveValue(int value) {
        if (value <= 0) {
            throw new IllegalArgumentException("value must be positive");
        }
        this.value = value;
    }
}
```

Java 25 allows certain statements before an explicit constructor invocation. The statements cannot use the instance under construction in ways prohibited by the language rules. This supports validation before superclass construction.

## JEP 507 — Primitive types in patterns (preview)

```
// Preview feature demonstration only.
// Compile and run with Java 25 preview enabled:
// javac --enable-preview --release 25 PrimitivePatternDemo.java
// java --enable-preview PrimitivePatternDemo

public class PrimitivePatternDemo {
    public static void main(String[] args) {
        Object value = Integer.valueOf(42);
        if (value instanceof int number) {
            System.out.println(number);
        }
    }
}
```

Preview features are not permanent API commitments. They require explicit enablement and may change or be removed in a later release. Do not use preview syntax in baseline course exercises unless the learning objective is specifically to study preview features.

# Appendix A. Common Java commands

```
# Check version
java --version
javac --version

# Compile
javac Hello.java

# Run a class
java Hello

# Run a source file directly
java Hello.java

# Compile with a specific release
javac --release 25 Hello.java

# Inspect bytecode
javap -c Hello

# Create a JAR
jar --create --file app.jar --main-class com.example.Main -C out .

# Run a JAR
java -jar app.jar

# Enable preview feature for Java 25
javac --enable-preview --release 25 Demo.java
java --enable-preview Demo
```

## Recommended project layout

```
java25-training/
  src/
    main/
      java/
        com/skynetix/training/
          Main.java
          model/
          service/
          repository/
    test/
      java/
        com/skynetix/training/
  README.md
  pom.xml
```

For multi-file and enterprise projects, use Maven or Gradle. Keep production code under src/main/java and tests under src/test/java. Use packages that reflect responsibility rather than creating one giant class.

# Appendix B. Java interview and revision checklist

- Explain JDK, JVM, bytecode, class loading and JIT compilation.
- Explain primitive types, wrapper classes, autoboxing and immutability.
- Explain equals/hashCode contract and why it matters in HashMap.
- Compare ArrayList, LinkedList, HashSet, TreeSet, HashMap and ConcurrentHashMap.
- Explain checked versus unchecked exceptions and try-with-resources.
- Explain generics, type erasure, bounded types and PECS.
- Explain lambda expressions, functional interfaces and stream laziness.
- Explain map, flatMap, reduce, groupingBy and parallel stream risks.
- Explain synchronized, volatile, locks, atomics and race conditions.
- Compare platform threads, executors and virtual threads.
- Explain heap, stack, metaspace, GC roots and memory leaks.
- Explain records, sealed classes, pattern matching and Java 25 language changes.
- Explain JDBC transactions, prepared statements and connection pooling.
- Explain unit testing, mocking, integration testing and test boundaries.
- Explain HTTP timeouts, retries, idempotency and API error handling.

# Appendix C. Official Java 25 references

**Oracle Java 25 language changes summary:**
https://docs.oracle.com/en/java/javase/25/language/java-language-changes-summary.html

**Oracle Java 25 language changes by release:**
https://docs.oracle.com/en/java/javase/25/language/java-language-changes-release.html

**Java SE 25 specifications:** https://docs.oracle.com/en/java/javase/25/docs/specs/index.html

**JDK 25 significant changes and migration guide:**
https://docs.oracle.com/en/java/javase/25/migrate/significant-changes-jdk-25.html

**Oracle Java 25 release announcement:** https://www.oracle.com/news/announcement/oracle-releases-java-25-2025-09-16/

**Java Tutorials and learning resources:** https://dev.java/learn/

**JUnit 5 user guide:** https://junit.org/junit5/docs/current/user-guide/

**Maven documentation:** https://maven.apache.org/guides/

**Gradle documentation:** https://docs.gradle.org/current/userguide/userguide.html

Always verify syntax, JDK flags and dependency versions against the documentation for the exact environment used in class. Java 25 is the target of this handbook; newer JDK releases may contain additional changes.